

1		5	
2		6	
3		7	
4			

%	NOTA
0-39	2
40-49	3
50-59	4
60-69	5
67-73	6
74-80	7
81-87	8
88-94	9
95-100	10

Ejercicio 1

Marca con una X si la instrucción modifica Registros y/o Memoria o nada. Al PC y los flags no se los considera registros. No puede hacer ninguna suposición acerca del contenido de los registros ni de la memoria.

ADD X0,X0,X0	R[☒] M[☒]	LDURB X0,[X0,#0]	R[☐] M[☒]
ADDIS X0,X0,#0	R[☒] M[☒]	LDURSW X0,[X0,#0]	R[☐] M[☒]
AND X31,X31,X31	R[☒] M[☒]	LSR X30,X30,#63	R[☒] M[☒]
ANDIS X0,X0,#0xFFF	R[☒] M[☒]	MOVZ X0,#0,LSL 0	R[☒] M[☒]
B 0	R[☐] M[☒]	ORRI X7,X7,#0	R[☒] M[☒]
BL 1	R[☐] M[☒]	STURB X30,[X30,#8]	R[☐] M[☒]
CBNZ X31, -1	R[☒] M[☒]	STURW X30,[X30,#16]	R[☐] M[☒]
EOR X8,X8,X8	R[☒] M[☒]	SUBI X1,X2,XZR	R[☒] M[☒]
LDUR X31,[X0,#16]	R[☐] M[☒]	SUBS X2,X2,XZR	R[☒] M[☒]

Ejercicio 2

Indicar exactamente cuántas veces se ejecuta la instrucción SUBI X0,X0,#1. Dejar el valor expresado con potencias y multiplicaciones.

```
MOVZ X1,#3,LSL#0
L0: MOVZ X0,#0,LSL#48
L1: SUBI X0,X0,#1
    CBNZ X0,L1
    SUBI X1,X1,#1
    CBNZ X1,L0
```

264, 3

Ejercicio 3

El siguiente programa en assembler LEGv8 pone a 0 la memoria en el rango [0x0000,0x1000)

```
.org 0xC0C0
MOVZ X0,#0x1000,LSL#0
L: SUBI X0,X0,#1
    STURB XZR,[X0,#0]
    CBNZ X0,L
```

?

- a) Indicar cuantas instrucciones ejecuta en total.
Dejar el valor expresado como sumas, multiplicaciones y/o potencias.

3 N + 2 Instrucciones

- b) Escribir al lado una versión modificada que ocupe exactamente la misma cantidad de instrucciones en memoria, pero que ejecute 2, 4 u 8 veces menos instrucciones (2, 4 u 8 veces más rápido).

Ejercicio 4

La siguiente secuencia de bytes es código de máquina LEGv8.

Desensamblar el programa, sabiendo que LEGv8 es LE (little-endian)

01 00 81 d2 21 04 00 d1 3f 00 00 38 c1 ff ff b5

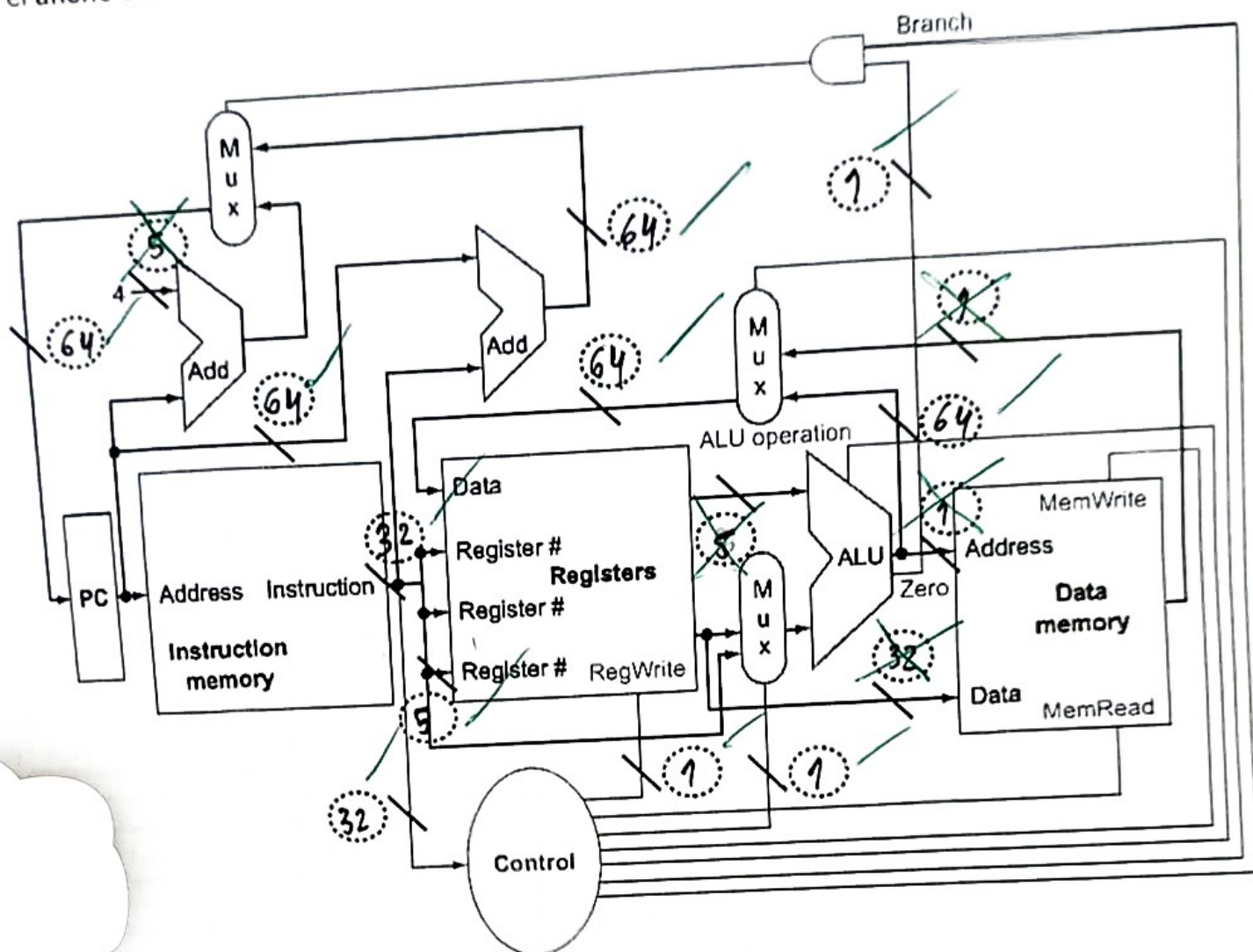
MOVZ	X1	#0	X100	LSL	#0
SUBI	X1	X1	#0	X10	
STUR	X1	[X1, #0]			
CBNZ	X1	L			

Para "C" con la siguiente relación entre variables y registros: x ← X0, dx ← X1.

de código correctamente formateado o penalidad por cada instrucción de más.

<pre> IF (X1 == 0) { X = X - 1; dx = dx + 1; } else { X = X + dx; } </pre>	<pre> CBNZ X0, ELSE SUBI X8, XZR, #1 EOR X1, X1, X8 ADDI X1, X1, #1 B EXIT ELSE: ADD X0, X0, X1 EXIT: </pre>
--	--

Ejercicio 6
 Marcar el ancho en bits de las 16 líneas de datos marcadas.

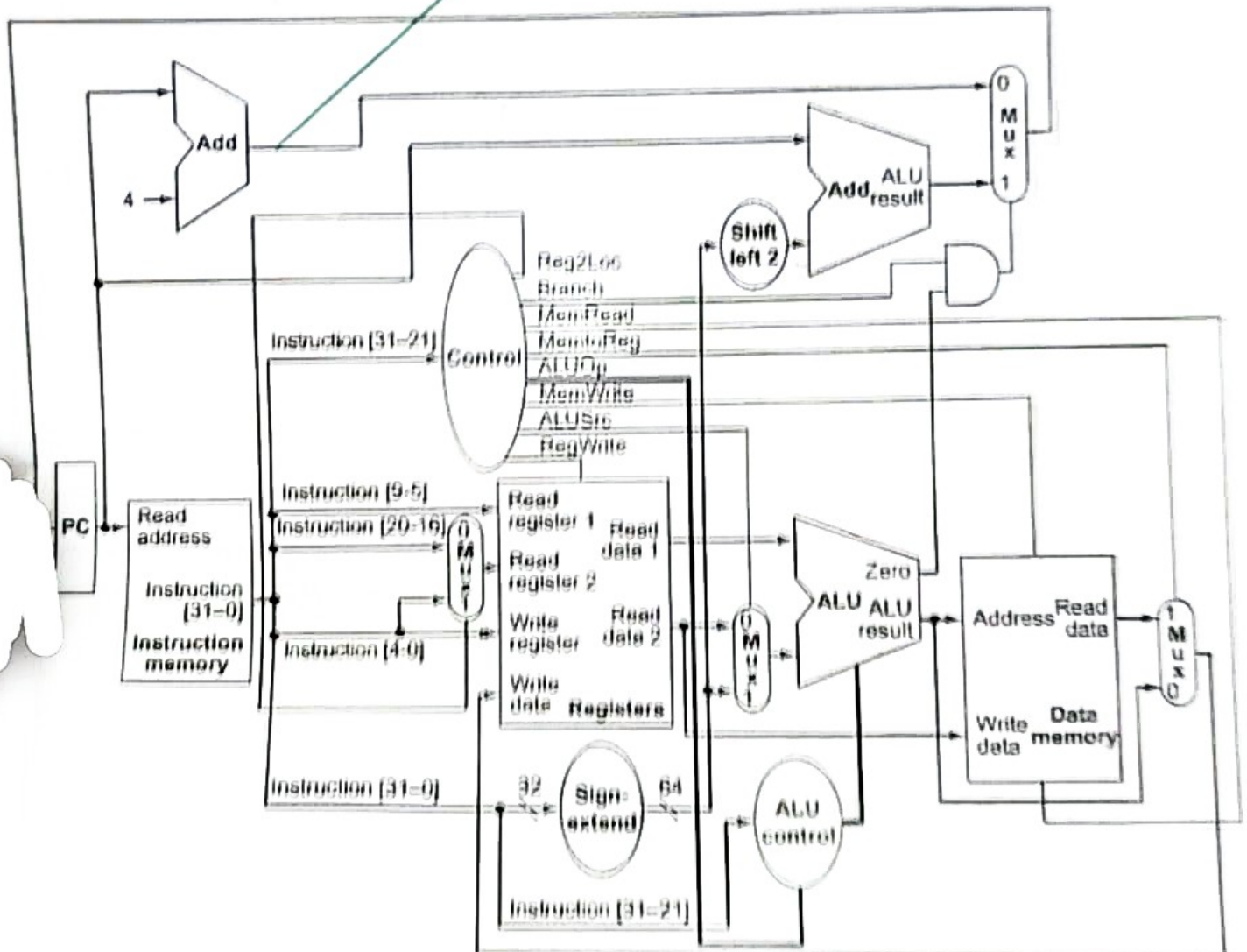


Ejercicio 7

Un opcode genera una instrucción no documentada que hace que Control tome estos valores. Abajo se muestra el diagrama completo.

Reg2Loc	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALU
1	0	1	1	1	0	1	PASS B

a) ¿Qué operación realiza?



1	5
2	6
3	7
4	

%	PUNTA
0-39	2
40-49	3
50-59	4
60-69	5
70-79	6
80-89	7
90-99	8
100-100	9

Ejercicio 1

Escribir un NOP, un skip, UNA instrucción que no hace nada. Una con cada nemónico.
 No se puede suponer nada acerca de los registros, la memoria, el PC o los flags.

Si no se puede, poner raya —.

ADDI	$xzr, xzr, \#0$	✓	LDURH	—	✓
ADDS	xzr, xzr, xzr	X	LSL	$xzr, xzr, \#0$	✓
ANDI	$xzr, xzr, \#0$	✓	MOVK	—	X
ANDS	xzr, xzr, xzr	X	ORR	xzr, xzr, xzr	✓
B	hilo de instrucciones 2. ej: B. done done.	✓	STUR	—	✓
BR	—	✓	STURH	—	✓
CBZ	—	X	SUB	xzr, xzr, xzr	✓
EORI	$xzr, xzr, \#0$	✓	SUBIS	$xzr, xzr, \#0$	X

Ejercicio 2

Completar el programa LEGv8 para que el loop itere exactamente 2^{64} veces.

ADDI $X_0, XZR, \#4295$

L0: SUBI $X_0, X_0, \#1$

CBNZ $X_0, L0$

X

Ejercicio 3

Escribir un programa LEGv8 que sobrescriba el rango de memoria $[0x0000, 0x1000)$ con 0s.
 Si usa más de 5 instrucciones, se baja el puntaje del ejercicio.

Ejercicio 4

Para el siguiente código LEGv8, la directiva `.org`, le indica al programa que ensambla la dirección de memoria donde posicionará el código.

a) Ensamblar el programa.

Ensamblador	Código de Máquina
<code>.org 0x2000</code>	
<code>ADD X16,XZR,XZR</code>	0x2000: 0x 2 B 1 F 0 3 F 0 ✓
<code>SUBI X0,XZR,#1</code>	0x2004: 0x D 1 0 0 0 7 E 0 ✓
<code>L: ADD X16,X16,X0</code>	0x2008: 0x 8 B 0 0 0 2 1 0 ✓
<code>CBNZ X16,L</code>	0x200C: 0x B 5 F F F F F 0 ✓

b) Dado que la arquitectura es LE (little-endian), escribir la secuencia de bytes en memoria:

0x2000: 0x 0 F ✗	0x2008: 0x 0 1
0x2001: 0x 3 0 ✗	0x2009: 0x 7 0
0x2002: 0x F 7	0x200A: 0x 0 0
0x2003: 0x B 2	0x200B: 0x B 2
0x2004: 0x 0 E	0x200C: 0x 0 F
0x2005: 0x 7 0	0x200D: 0x F F
0x2006: 0x 0 0	0x200E: 0x F F
0x2007: 0x 7 0	0x200F: 0x 5 B

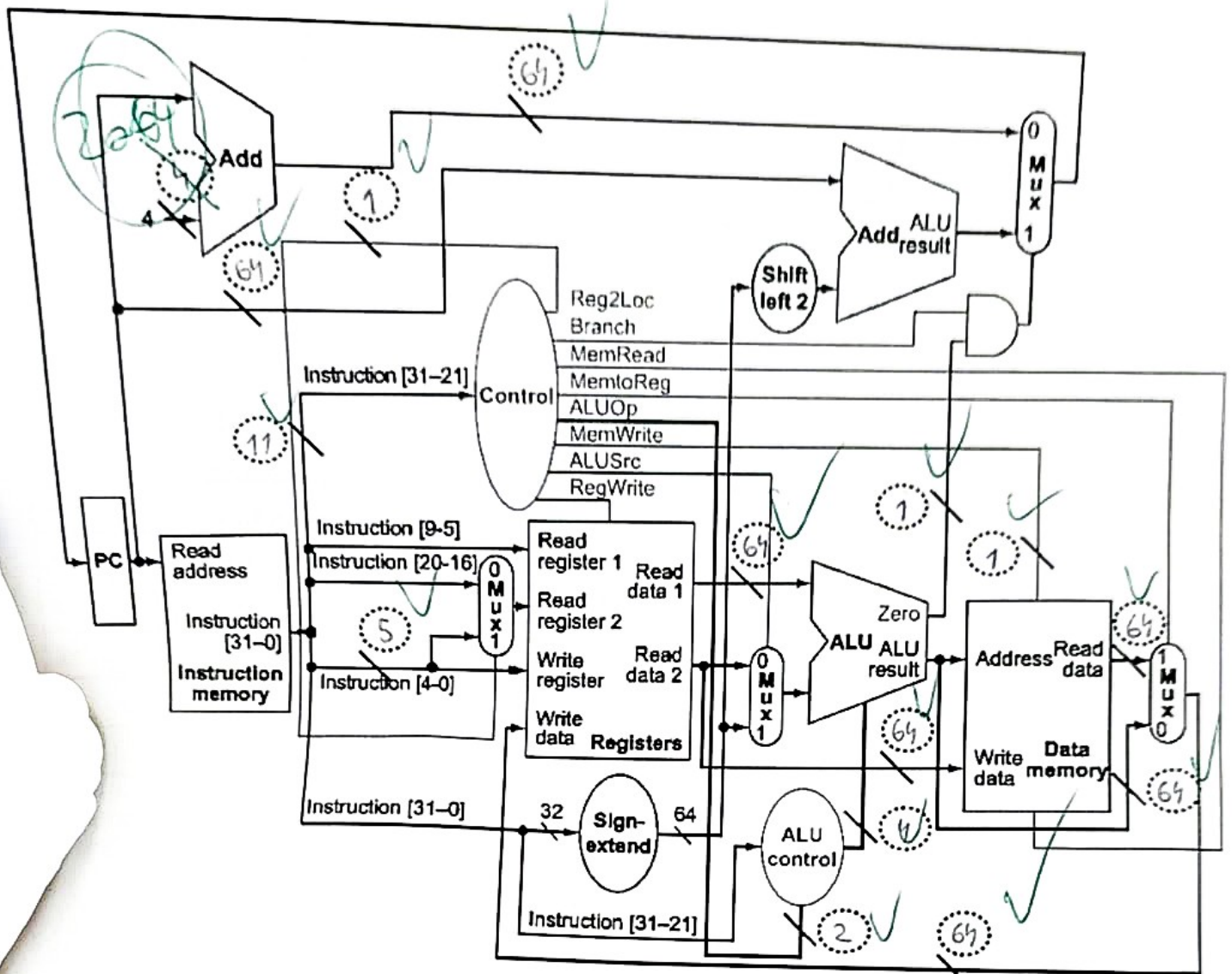
Ejercicio 5

Compilar a LEGv8 con la siguiente relación entre variables y registros: `x0 ← x0`, `dx ← x1`.
En 7 instrucciones o penalidad por cada instrucción de más.

<pre> if (x==0) # rebota dx = -dx; else # avanza x = x+dx; </pre>	<pre> ADDI X0,X0,#0 CBZ X0,rebota ADD X0,X0,X1 B.done rebota: SUB X1,X2R,X1 done: </pre>
--	--

Ejercicio 6

Marcar el ancho en bits de las 15 líneas de datos marcadas.



Ejercicio 7

Completar la tabla de Control para que el Data Path ejecute la instrucción STUR.

Reg2Loc	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
1	1	X	0	0	1	0	00

En cada casilla poner {0, 1, X}. En la ALUOp poner el nombre de la operación, no hace falta el código.